

GamutX

Custom Packages Fundamentals

PYTHON SCRIPTING

Module vs Package vs Library

- Python Module
- Python Package
- Python Library
- Package vs. Library
- Custom Package vs. Custom Library

Python Module

- A **module** is a **single Python** file (ending in **.py**) that contains code — such as **functions**, **classes**, and **variables**
- which you can **import and use** in another **Python program**.

Example:

example_module.py

```
example_module.py > greet
1 def greet():
2     return "Hello, World!"
```

added a simple function

test.py

```
test.py
1 import example_module
2
3 print(example_module.greet())
4
5 # Output: Hello, World!
```

imported module

call greet() Func

- **example_module.py** is the module.
- We can **import** and **reuse** it in other files.

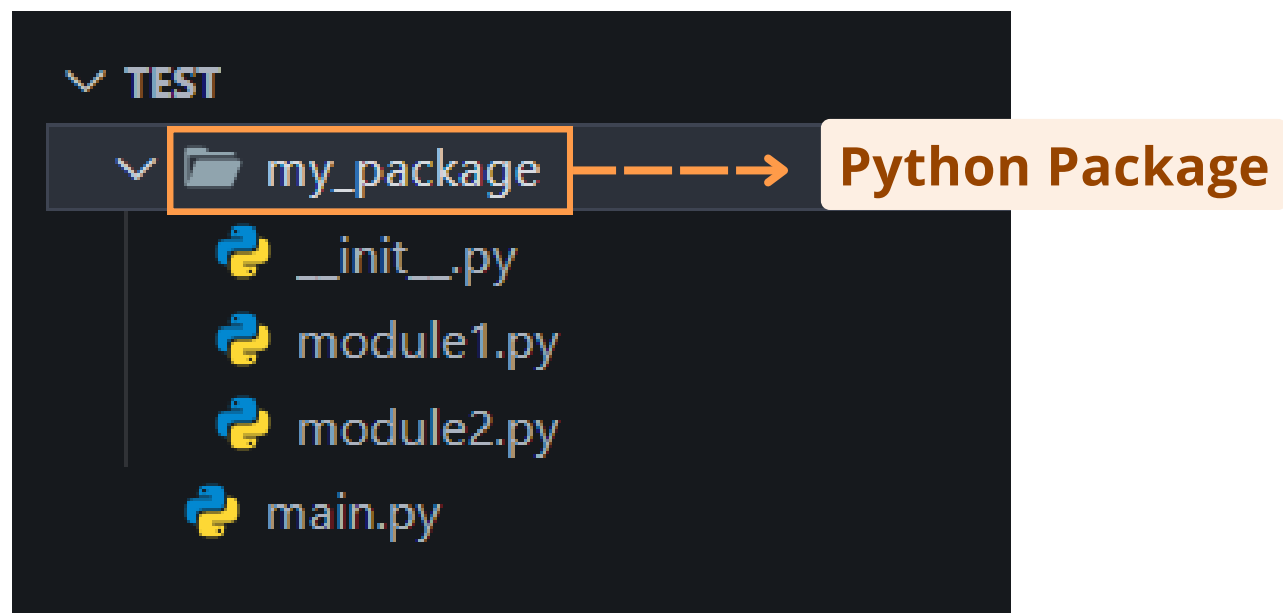
Purpose:

- To **organize code** into **reusable components**.
- Helps avoid repeating code across **multiple scripts**.

Python Package

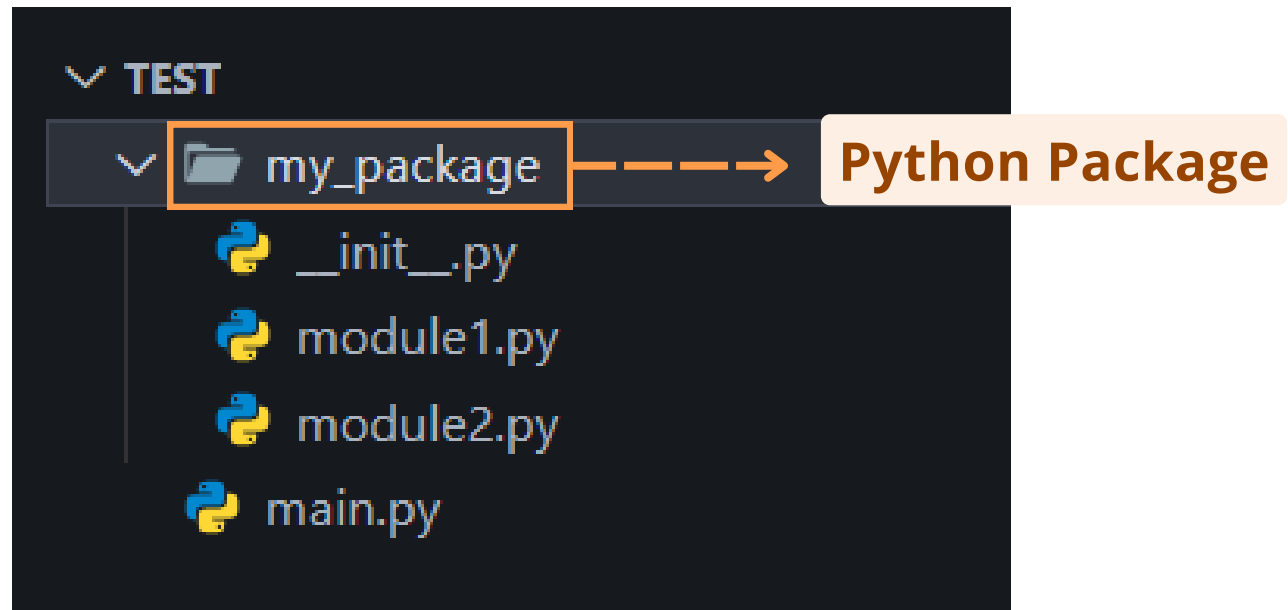
- A **package** is a **directory (folder)** that contains **multiple Python modules**.
- To be recognized as a package, this folder **must include a special file called `__init__.py`**.
- This file can be empty or contain some **initialization code**.

Folder Structure:



Now let's look at what's inside each module:

Folder Structure:



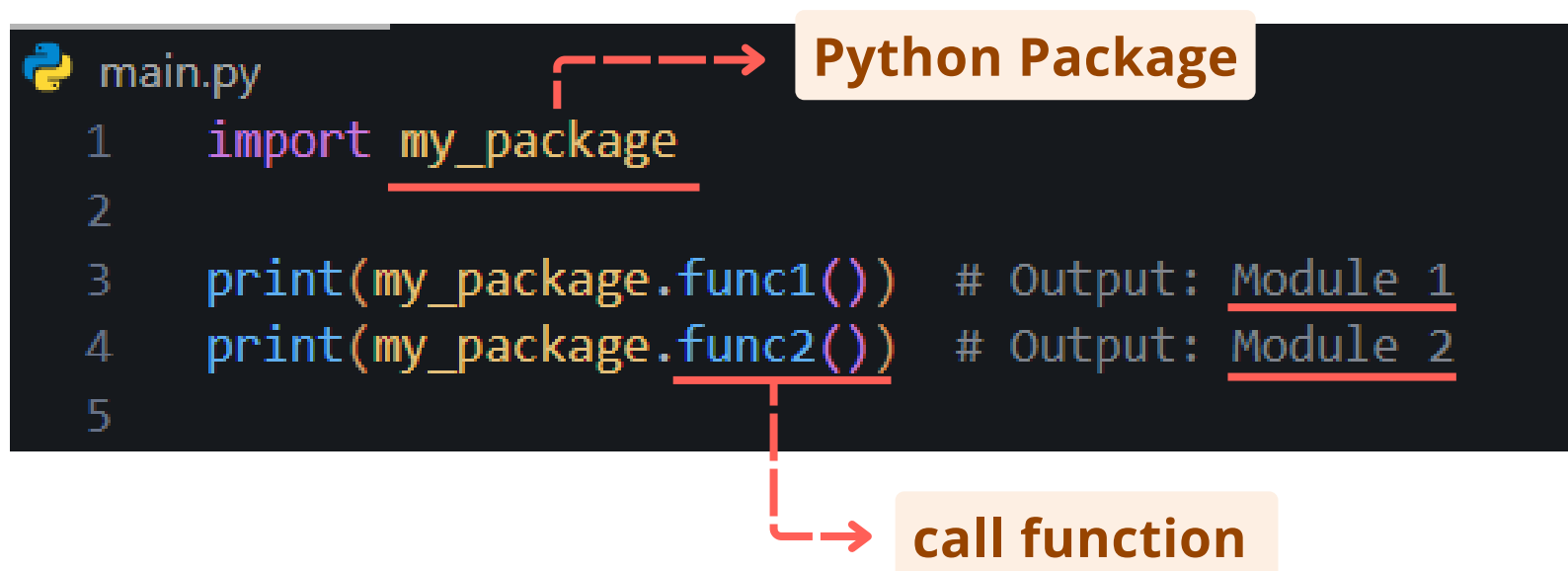
File Content

```
# my_package/module1.py
def func1():
    return "Module 1"

# my_package/module2.py
def func2():
    return "Module 2"

# my_package/__init__.py
from .module1 import func1
from .module2 import func2
```

main.py



Purpose:

- Organizes **multiple related modules** under a **common namespace**.
- Makes large projects **easier** to **maintain**.

Python Library

A **library** is a **collection of packages ,modules** bundled together to provide a **set of related functionality**.

Example:

```
main.py > ...  
1  import requests  
2  
3  response = requests.get('https://jsonplaceholder.typicode.com/todos/1')  
4  
5  print(response.json())  
6  # Output: {'userId': 1, 'id': 1, 'title': 'delectus aut autem', 'completed': False}  
7
```

Annotations in the code:

- import library** points to the `import requests` line.
- Send GET request** points to the `requests.get()` method call.
- Parse JSON data** points to the `response.json()` method call.

Liink <https://jsonplaceholder.typicode.com/todos/1>

The **requests** library is composed of multiple modules and packages to handle **HTTP requests**.

Purpose:

- Libraries save development time by providing ready-made tools
 - for tasks like **HTTP requests, image processing, data analysis, GUI creation,**
 - and **file handling** (useful in VFX pipelines).

Summary Table

Feature	Type	Contains	Contains
• Module	→ .py file	→ Functions, classes, variables	→ Reusable code in one file
• Package	→ Folder + __init__.py	→ One or more modules	→ Organize code into namespaces
• Library	→ Distribution	→ Modules + packages	→ Bundled tools for specific tasks

Rule of Thumb:

- If we're talking about **how the code is organized** → it's a **package**.
- If we're talking about **what the code does or solves** → it's a **library**.

Package vs. Library

In Python, the terms "**package**" and "**library**" are often used **interchangeably** in casual conversation, but they have slightly different meanings.

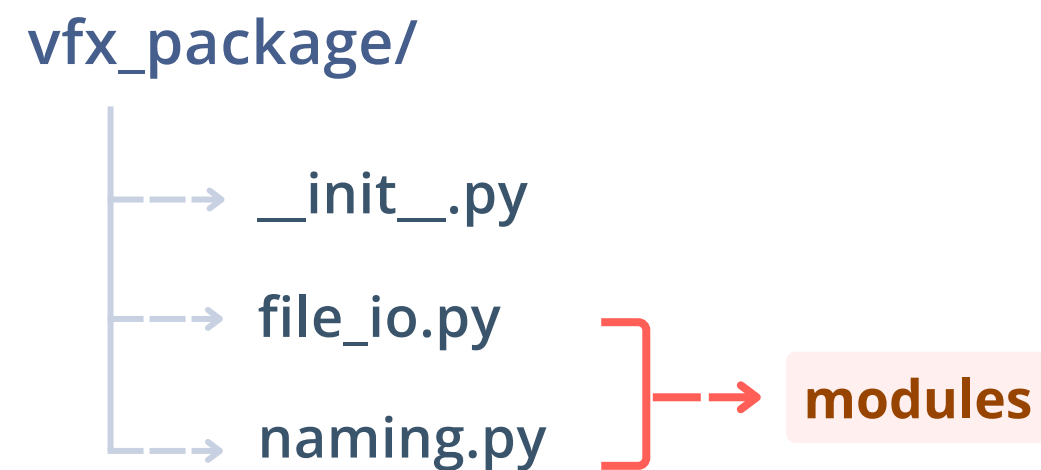
Feature	Package	Library
• What it is	→ A folder of Python modules	→ A toolset made up of modules/packages
• Structure	→ Contains <code>__init__.py</code> + modules	→ Can contain packages, modules, metadata
• Purpose	→ Organize code within a project	→ Solve specific problems, reusable across projects
• Example	→ <code>my_package/</code>	→ <code>requests</code> , <code>Pandas</code> , <code>PyQt5</code>
• Common Use Case	→ Internal project structure	→ External functionality (e.g., <code>GUI</code> , <code>HTTP</code> , <code>math</code>)
• Installation	→ Not installed separately	→ Installed via <code>pip</code> or <code>conda</code>

Custom Package vs. Custom Library

What is a Custom Package?

- A **custom package** is a folder **we create** to organize our Python code (modules).
- It contains one or more **Python modules** and an **`__init__.py`** file.

Folder Structure



Used like:

```
from vfx_package import file_io
file_io.load_sequence('/renders')
```

Custom Package

module

Package = `__init__` file + Modules

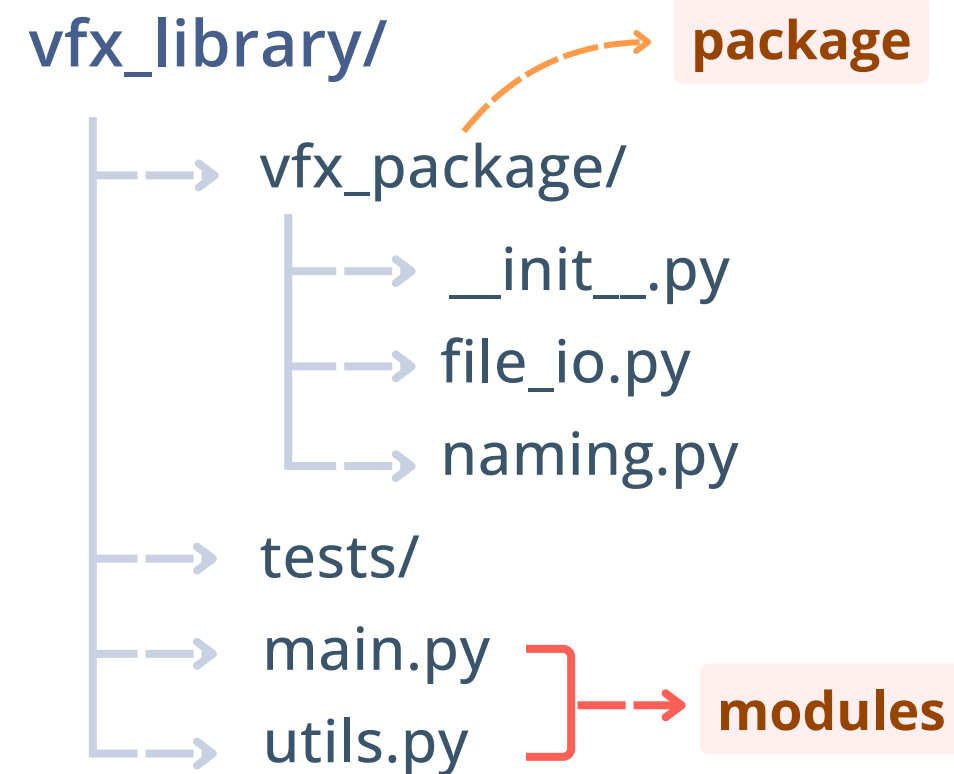
Purpose:

- Structure code within a **single project**
- Keep related functions and classes in separate files

What is a Custom Library?

- A **custom library** is a **reusable collection** of packages and modules.
- It's designed to be **shared** and **reused** across multiple projects or by a team.

Folder Structure



Key Characteristics:

- May include **multiple packages**
- **Install locally** → `pip install`
- **Use in a project** → `from vfx_package.naming import clean_name`

Summary

Term	Focus	Reusability	Example Use
• Custom Package	→ Internal structure	→ One project	→ Splitting logic in a tool
• Custom Library	→ Reusable toolkit	→ Multiple projects	→ Team-wide utilities

Rule of Thumb

- If you're organizing related code: you're making a **custom package**.
- If you're bundling tools for reuse and distribution: you're making a **custom library**.

In practice, a **custom library** is made up of one or more **custom packages**.

Working with Custom Packages in Python

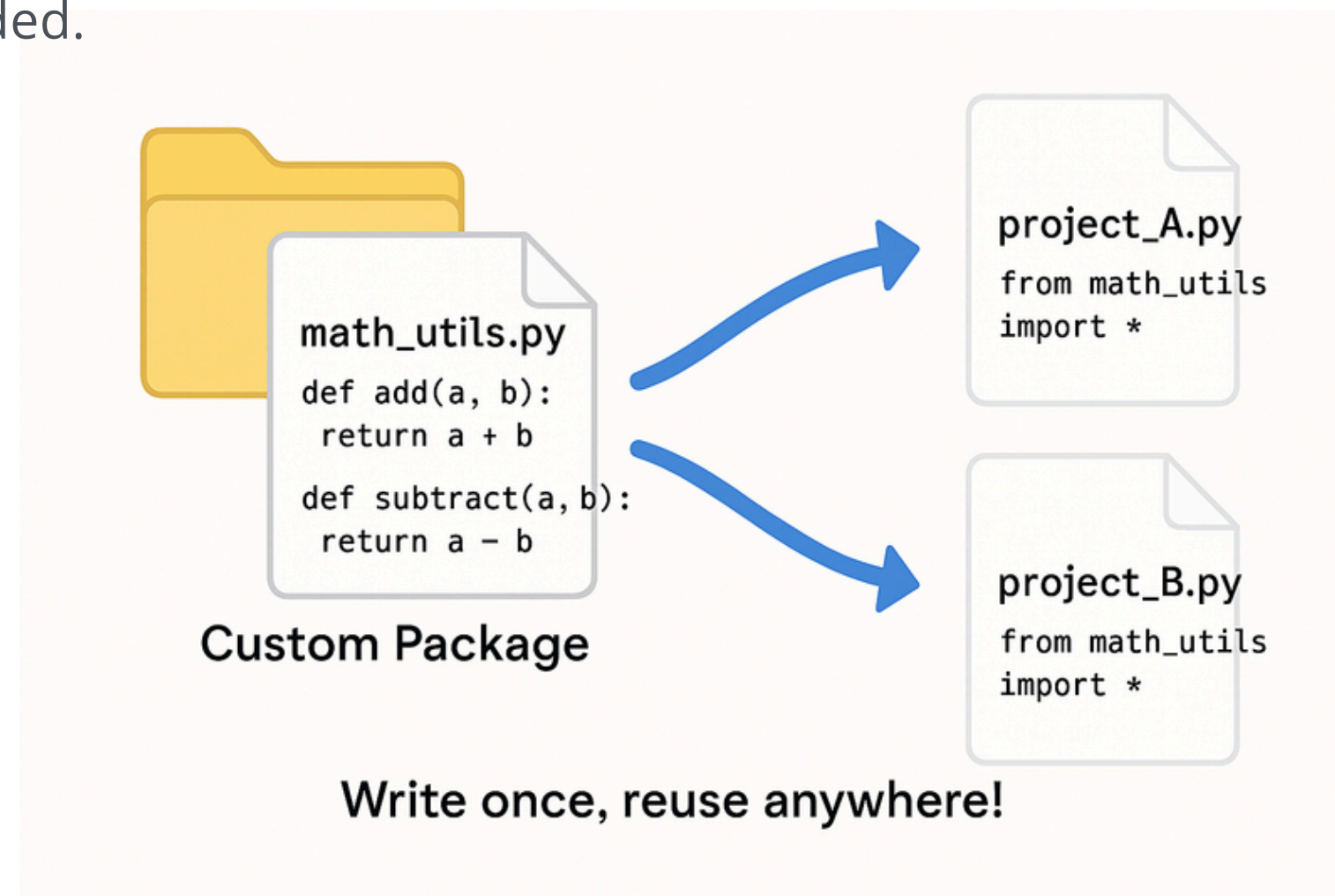
- What are Custom Packages?
- Create a Custom Package
- What is `__init__.py`?
- Importing the Package

What are Custom Packages?

- A **custom package** is a folder **we create** to organize our Python code (modules).
- It contains one or more **Python modules** and an **`__init__.py`** file.

For example:

- you might have **several functions** in a file that you want to **reuse** in **multiple** projects.
- Instead of writing the same functions again and again, you can save them in a **custom library** and import it whenever needed.



Create a Custom Package

Create the Folder Structure

Make a folder named **my_package** inside **Dev Folder** and add these files:

```
my_package/  
├── __init__.py  Required  
├── module1.py  
├── module2.py  
├── module3.py  
└── module4.py
```

__init__.py is **required** for Python to recognize this folder as a package. It can be empty or **contain initialization code**.

Add Code to Each Module

module1.py

```
name = "Module 1"
value = 10

def greet():
    return "Hello from Module 1!"

def add(a, b):
    return a + b
```

module2.py

```
name = "Module 2"
factor = 2

def multiply(a, b):
    return a * b

def divide(a, b):
    return a / b
```

module3.py

```
name = "Module 3"
status = "Active"

def status_check():
    return f"Status: {status}"
```

module4.py

```
name = "Module 4"
version = "1.0.0"

def get_version():
    return f"Version: {version}"
```

What is __init__.py?

__init__.py

```
print("Initializing my_package")
```

- __init__.py is a **special Python file** used to indicate that a directory should be treated as a package.
- It must be **placed inside a folder** that we want Python to **recognize as a package**.

Importing the Package

We've built a custom package (**my_package**), and we want to use it in our Python script (**main.py**).

```
from my_package import module1
print(module1.greet())
# "Hello from Module 1!"
```

import module

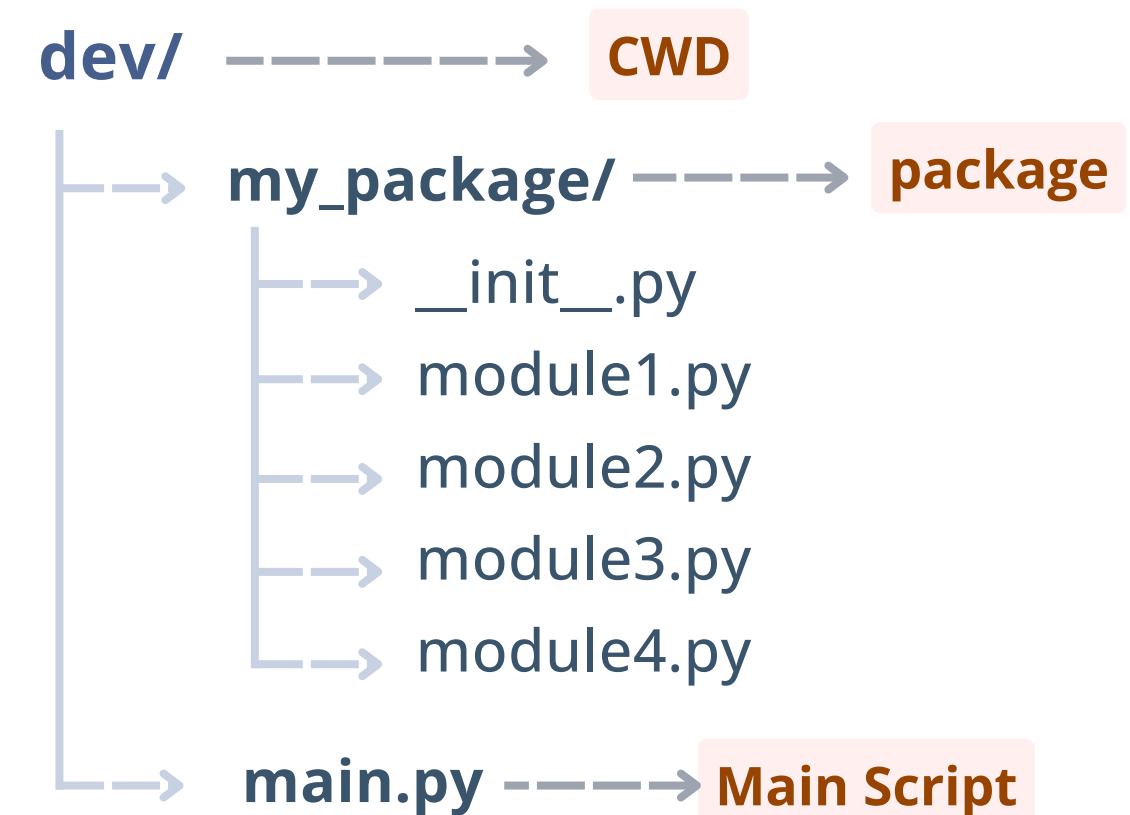
When we run, we get No error it works.

Why?

Because the **package is in the same directory as the script**.

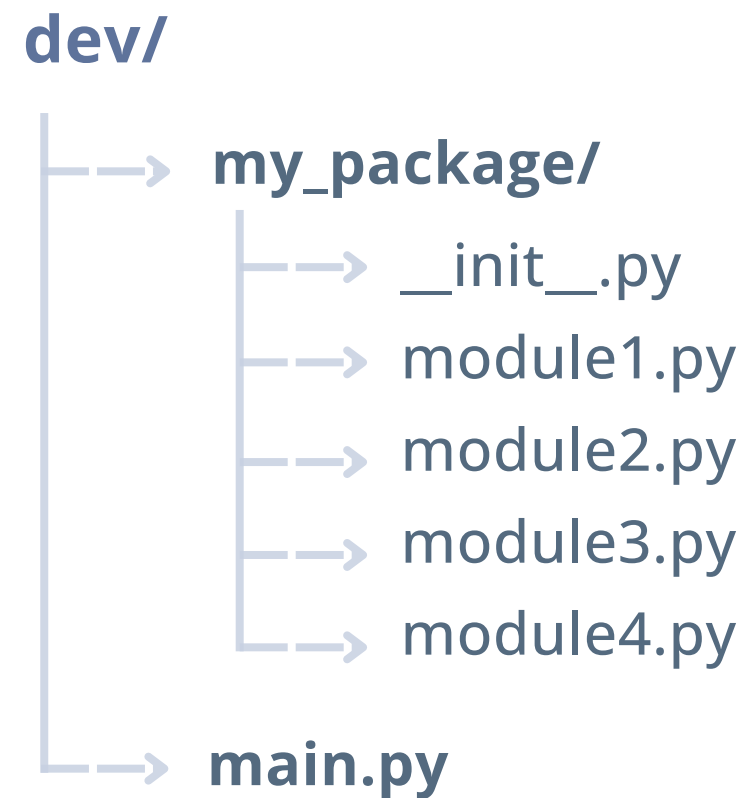
- Python knows the **current directory of the script when it's executed**.
- It automatically checks that location for imports.

Folder Structure

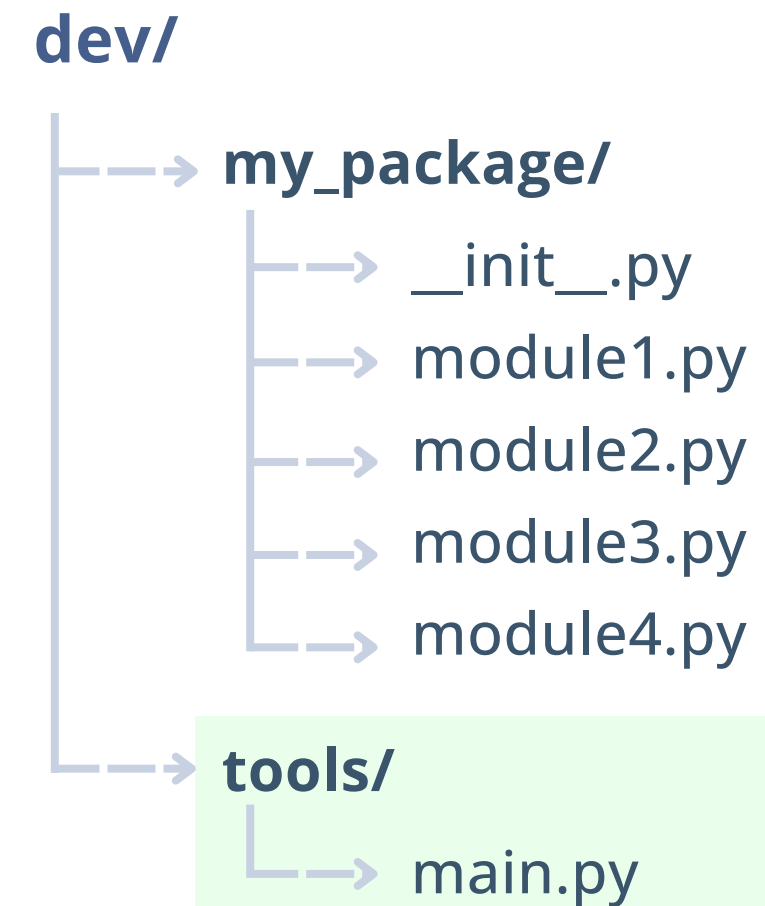


Now, Let's **Move main.py** to Another Folder

Old Folder Structure



Updated Folder Structure



Move inside a Tools Folder

Now that we've moved **main.py** into the **Tools** folder, let's **run the program** and check if the **import statements** are **Work** or **not**

main.py

```
tools > 🐍 main.py
1  from my_package import module1
2
3  print(module1.greet())
4
```

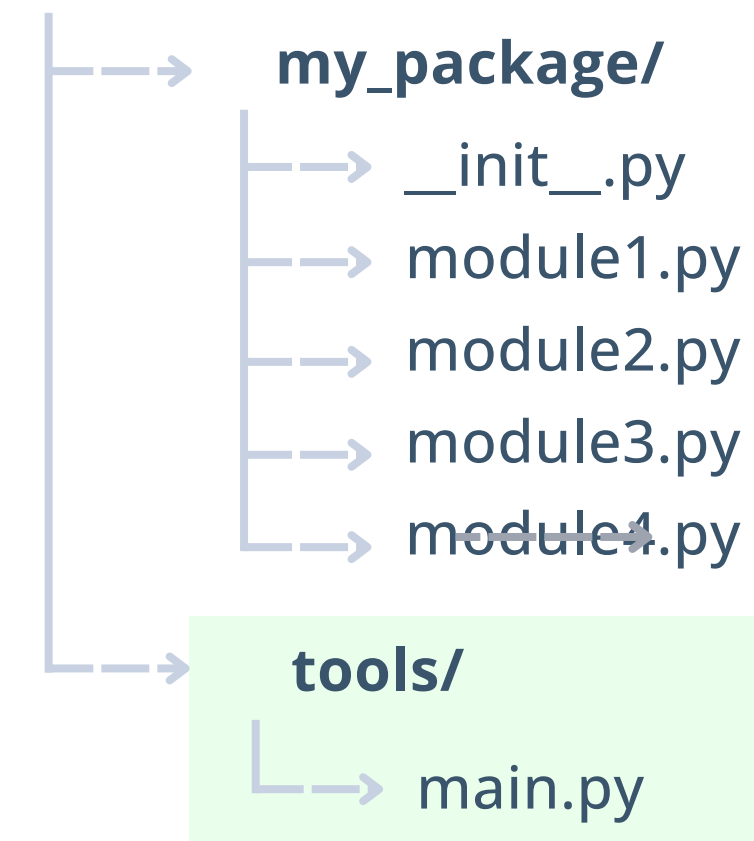
trying to import package

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\pralhad\Desktop\dev> & C:/Users/pralhad/AppData/Local/Programs/Python/Python38-32/Python.exe main.py
Traceback (most recent call last):
  File "c:\Users\pralhad\Desktop\dev\tools\main.py", line 1, in <module>
    from my_package import module1
ModuleNotFoundError: No module named 'my_package'
PS C:\Users\pralhad\Desktop\dev>
```

get ModuleNotFoundError Error

Updated Folder Structure

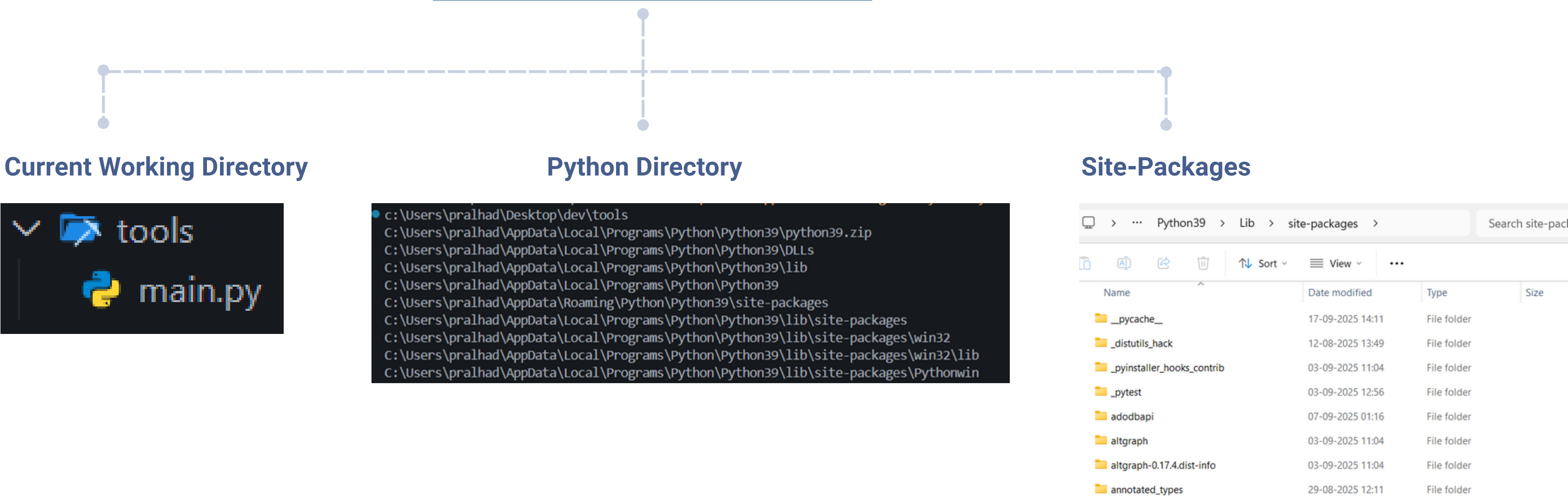
dev/



Why Did It Fail ?

Python couldn't find **my_package** because it's **not in the list of directories** that Python searches when we try to import something.

This list is stored in `sys.path`. By default, `sys.path` includes:



Python\Python39\Lib\site-packages



So how do we tell Python where to find
our custom package?

Importing Python Packages from Custom Location

- Understanding sys.path
- Add Your Custom Package Path to sys.path
- Verify If Package Path is Added
- Importing Custom Packages

Understanding sys.path

Python uses the **sys.path** list — provided by the built-in sys module — to know **where to look** for packages and modules.

What Is sys.path?

- **sys.path** is a list of directory paths.
- Python checks each path in this list when we use **import**.

Printing sys.path

```
tools >  main.py
1  import sys
2
3  print(sys.path)
4
```

→ Gives list of paths where Python searches for modules

↓
path of Directory

```
PS C:\Users\pralhad\Desktop\dev> & C:/Users/pralhad/AppData/Local/Programs/Python/Python39/python.exe c:/Users/pralhad/Desktop/dev/tools/main.py
• ['c:\\Users\\pralhad\\Desktop\\dev\\tools', 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\python39.zip', 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\DLLs', 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\lib', 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39', 'C:\\Users\\pralhad\\AppData\\Roaming\\Python\\Python39\\site-packages', 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\lib\\site-packages', 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\lib\\site-packages\\win32', 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\lib\\site-packages\\Pythonwin']
○ PS C:\Users\pralhad\Desktop\dev>
```

Check where Python is looking for modules

You can print the **sys.path** list to see where Python searches for modules:

```
tools >  main.py > ...  
1  import sys  
2  
3  for path in sys.path:  
4      print(path)  
5
```

Loop through system paths

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\pralhad\Desktop\dev> & C:/Users/pralhad/AppData/Local/Programs/Python/Python39.  
● c:\Users\pralhad\Desktop\dev\tools  
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\python39.zip  
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\DLLs  
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib  
C:\Users\pralhad\AppData\Local\Programs\Python\Python39  
C:\Users\pralhad\AppData\Roaming\Python\Python39\site-packages  
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib\site-packages  
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib\site-packages\win32  
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib\site-packages\win32\lib  
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib\site-packages\Pythonwin  
○ PS C:\Users\pralhad\Desktop\dev>
```

list of all the directories that Python checks
when trying to import a module.

Add Your Custom Package Path to sys.path

Since our custom package (**my_package**) is in a different location (dev/), we need to add its path **manually** before importing.

Option 1: Direct Append

```
tools >  main.py
1  import sys
2
3  sys.path.append(r"C:\Users\pralhad\Desktop\dev")
4
```

Hardcode the folder path

Option 2: Using os.path.abspath()

```
tools >  main.py > ...
1  import sys
2  import os
3
4  package_path = os.path.abspath("../dev")
5
6  print(package_path)
7  # Output: C:\Users\pralhad\Desktop\dev
8
9
10 if package_path not in sys.path: -->
11     sys.path.append(package_path)
12
```

Get absolute path dynamically

Avoid duplicate entries

Add folder to search path

Verify If Package Path is Added

Now, let's print the `sys.path` list again to confirm that our custom folder has been added:

```
tools >  main.py > ...
1  import sys
2
3  sys.path.append(r"C:\Users\pralhad\Desktop\dev")
4
5  for path in sys.path:
6      print(path)
```

 Loop through system paths

Output

```
● c:\Users\pralhad\Desktop\dev\tools
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\python39.zip
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\DLLs
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib
C:\Users\pralhad\AppData\Local\Programs\Python\Python39
C:\Users\pralhad\AppData\Roaming\Python\Python39\site-packages
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib\site-packages
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib\site-packages\win32
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib\site-packages\win32\lib
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib\site-packages\Pythonwin
C:\Users\pralhad\Desktop\dev
○ PS C:\Users\pralhad\Desktop\dev>
```

If we see the **dev/** path, we're good to go.

Importing Custom Packages

Using Direct Append

```

tools > main.py
1  import sys
2
3  sys.path.append(r"C:\Users\pralhad\Desktop\dev")
4
5
6  from my_package import module1, module2, module3, module4
7
8  print(module1.greet())
9  print(module1.add(5, 3))
10
11  print(module2.multiply(4, 2))
12  print(module2.divide(8, 2))
13
14  print(module3.status_check())
15  print(module4.get_version())
16
17  ...
18  # Output:
19  Initializing my_package
20  Hello from Module 1!
21  8
22  8
23  4.0
24  Status: Active
25  Version: 1.0.0
26  ...

```

Adds path Manually

imports multiple modules

Calls get_version() from module4

Using os.path.abspath()

```

import sys
import os

package_path = os.path.abspath("../dev")

print(package_path)
# Output: C:\Users\pralhad\Desktop\dev

if package_path not in sys.path:
    sys.path.append(package_path)

from my_package import module1, module2, module3, module4

print(module1.greet())           # Hello from Module 1!
print(module1.add(5, 3))         # 8

print(module2.multiply(4, 2))    # 8
print(module2.divide(8, 2))      # 4.0

print(module3.status_check())    # Status: Active
print(module4.get_version())     # Version: 1.0.0

```

Builds the absolute path dynamically

prevents duplicate path entries

Note:

- we need to append the path every time you run the **main file**
- because **sys.path.append()** only affects the current Python session.
- Once the **script ends**, the **path is forgotten**.

Adding Imports in `__init__.py`

- Why is `__init__.py` Required?
- Adding Imports in `__init__.py`
- Why we did Changes?
- Using wildcard import *
- Using `__all__`
- What is `__all__`?
- Best Practice for `__init__.py` Imports

Why is __init__.py Required?

Let's see an Example

```
tools > main.py
1  import sys
2
3  sys.path.append(r"C:\Users\pralhad\Desktop\dev")
4
5  import my_package
6
7
8  # Output: Initializing my_package
9
```

Imports the custom package

__init__.py File

```
my_packages > __init__.py
1  print("Initializing my_package")
2
```

When imported, the package's **__init__.py** file runs automatically

Purpose	Description
• Marks a folder as a package	→ Without this file, Python treats the folder as a regular directory.
• Allows package initialization	→ Code inside __init__.py runs when the package is imported.
• Controls top-level imports	→ Determines what gets exposed when we use <code>from my_package import *</code> .

Now, let's see what happens when we **add imports in the `__init__.py`** file
&
how it **simplifies our workflow.**



Adding Imports in __init__.py

Let's say we have the following package:

```
my_package/  
├── __init__.py  
├── module1.py (greet, add)  
├── module2.py (multiply, divide)  
├── module3.py (status_check)  
└── module4.py (get_version)
```

this are **functions & variables** inside
respective module

Previously, we had:

```
my_package >  __init__.py  
1 print("Initializing my_package")
```

Now, we update it to expose selected functions directly:

Updated __init__.py

```
my_package >  __init__.py  
1 print("Initializing my_package")  
2  
3 from .module1 import greet, add  
4 from .module2 import multiply, divide  
5 from .module3 import status_check  
6 from .module4 import get_version
```

imported function and Variables at Package Level

Key Changes in `__init__.py`

Aspect	Before	After
• Import Syntax	→ from my_package import module1	→ from my_package import greet
• Code Readability	→ Less intuitive	→ More direct and readable
• Top-Level Access	→ Only modules accessible	→ Specific functions accessible
• Wildcard Import	→ Nothing exposed	→ Only explicitly imported items exposed

Why we did Changes?

Easier Imports at the Package Level

Before:

module imports

```
from my_package import module1, module2  
  
print(module1.greet())  
print(module2.multiply(4, 2))
```

Each function had to be accessed through its module

After:

Custom package

Functions

```
from my_package import greet, multiply  
  
print(greet())           # Hello from Module 1!  
print(multiply(4, 2))    # 8
```

- Functions are exposed directly at the package level,
- allowing you to import them straight from my_package.

Key Benefit:

- Cleaner and more **intuitive** imports.
- No need to **repeatedly** reference **individual** modules.

Easier Imports at the Package Level

By explicitly defining what functions and variables are exposed at the package level in `__init__.py`,

- it's **easier** for other **developers to understand what the package offers**.
- For example, `__init__.py` now acts as an **API entry point for the package**.

Explicit Control Over Exports

- Using imports in `__init__.py`, you can **control what is accessible at the top level of the package**.
- If we **only import specific functions**,
 - others **won't be able to access internal or helper functions** from the modules **directly**.

Using wildcard import *

__init__.py

```
my_package >  __init__.py
1 print("Initializing my_package")
2
3 from .module1 import greet
4
```

Only import greet function

main.py

```
tools >  main.py
1 import sys
2
3 sys.path.append(r"C:\Users\pralhad\Desktop\dev")
4
5
6 from my_package import *
7
8 print(greet())
9
10 print(add(4, 6))
```

imports only what's added in __init__.py

Error! add() not imported

```
PS C:\Users\pralhad\Desktop\dev> & C:/Users/pralhad/AppData/Local/Programs/Python/Python39-64/Python.exe
⊗ Initializing my_package
→ Hello from Module 1!
Traceback (most recent call last):
  File "c:\Users\pralhad\Desktop\dev\tools\main.py", line 10, in <module>
    print(add(4, 6))
NameError: name 'add' is not defined
○ PS C:\Users\pralhad\Desktop\dev>
```

How It Work

When we use the **wildcard** import:

```
from my_package import *
```

Python will only import:

- Names **explicitly imported** in **`__init__.py`**
- Names **listed** in a **special variable** called **`__all__`**

Using __all__

Only One Function in __all__

__init__.py

Imports both functions

```
from .module1 import greet, add  
  
__all__ = ["greet"]
```

special Variable

Only greet() is accessible when using *

main.py

```
import sys  
import os  
  
package_path = os.path.abspath("../dev")  
  
if package_path not in sys.path:  
    sys.path.append(package_path)  
  
from my_package import *  
  
print(greet())    # Hello from Module 1!  
  
print(add(3, 4))  
# Traceback (most recent call last):  
#   File "c:\Users\pralhad\Desktop\dev\tools\main.py", line 16, in <module>  
#     print(add(3, 4))  
# NameError: name 'add' is not defined
```

Imports only what's listed in __all__

Works (allowed in __all__)

add() not included in __all__

Why do we still need import statements if we're using `__all__`?

Let's say we write this in `__init__.py`:

```
__all__ = ["get_version"]
```



But we **do not** include this line:

```
from .module4 import get_version
```



Required to load the function into the package

when we run: code in main

```
from my_package import *
```

→ Tries to import what's in `__all__`

```
print(get_version())
```

```
# Traceback (most recent call last):
```

```
#   File "c:\Users\pralhad\Desktop\dev\tools\main.py", line 12, in <module>
```

```
#     from my_package import *
```

```
# AttributeError: module 'my_package' has no attribute 'get_version'
```

→ Fails because `get_version` was never imported in `__init__.py`

Import All Public Functions Using `__all__`

Now let's update our package to **make all key functions from every module** accessible using `*`.

`__init__.py`

Imports functions from all modules

```
from .module1 import greet, add
from .module2 import multiply, divide
from .module3 import status_check
from .module4 import get_version
```

```
__all__ = [
    "greet", "add",
    "multiply", "divide",
    "status_check",
    "get_version"
]
```

Lists all public functions

Makes every key function accessible when using `*`

`main.py`

```
import sys
import os
```

```
package_path = os.path.abspath("../dev")
```

```
if package_path not in sys.path:
    sys.path.append(package_path)
```

```
from my_package import *
```

Imports all public functions

```
print(greet())           # Hello from Module 1!
print(add(2, 3))         # 5
print(multiply(4, 5))    # 20
print(divide(10, 2))     # 5.0
print(status_check())    # Status: Active
print(get_version())     # Version: 1.0.0
```

- `__all__` provides a clean interface
- only the intended public functions are imported.

What is `__all__`?

- `__all__` is a **special variable** used inside a package's `__init__.py` file.
- It must be assigned a **list of strings**
 - where **each string is the name of a function, class, or variable**
 - we want to expose when someone uses a **wildcard import** like:

```
from my_package import *
```

Think of `__all__` as a filter:

- It limits what gets imported when using *
- Anything not listed in `__all__` will not be accessible through wildcard import — even if it was imported inside `__init__.py`

Best Practice for `__init__.py` Imports

- Only expose **public functions** or **classes** in `__init__.py`.
- Avoid importing everything (*) to **maintain clarity**.
- Use **explicit imports** to make the package **API clean**.

Key Points

- We control **exactly what gets exposed** when someone imports the package
- Wildcard imports don't leak internal or unnecessary functions
- This helps keep our codebase **clean**, **predictable**, and **secure**

More About Custom Packages

- Understanding Functions in Custom Module
- Multi-Level Packages
- Flat vs Multi-Level Packages
- When to Use Each
- Avoid Hardcoding Paths When Importing

Understanding Functions in Custom Module

Inspecting Our Custom Package with dir()

- We've already created a custom package named **my_package**, and we've added its path to **sys.path** earlier in the project.
- Now, in our **main.py**, we can directly import and inspect it:

```
tools > main.py
1  import sys
2
3  sys.path.append(r"C:\Users\pralhad\Desktop\dev")
4
5
6  import my_package
7  import my_package.module1 as mod1
8
9  print(dir(my_package))
10
11  print()
12
13  print(dir(mod1))
14
```

inspect what's inside the package

Shows all attributes available in my_package

```
['__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
 '__name__',
 '__package__', '__path__', '__spec__', 'greet', 'module1']
```

Inspect what's inside module1

```
['__builtins__', '__cached__', '__doc__', '__file__',
 '__loader__', '__name__', '__package__', '__spec__',
 'add', 'greet', 'name', 'value']
```

Shows all functions and variables defined inside module1

Direct Access to Functions via `__init__.py`

If we define imports like this in `my_package/__init__.py`:

```
from .module1 import greet, add
from .module2 import multiply, divide
from .module3 import status_check
from .module4 import get_version
```

import functions

Then, in `main.py`, we can access all those functions directly:

```
print(my_package.greet())
print(my_package.add(5, 3))
```

Accessing function

Benefit:

- We don't need to import from each module individually
- This results in cleaner, flatter imports
- The package behaves more like a professional, ready-to-use library

Summary

Concept

- **Import Syntax**

Without Imports in `__init__.py`

→ from **my_package**.module1 import greet

With Imports in `__init__.py`

→ from **my_package** import greet

- **Function Access**

→ Indirect via module

→ Direct via package

- **Inspection (`dir`)**

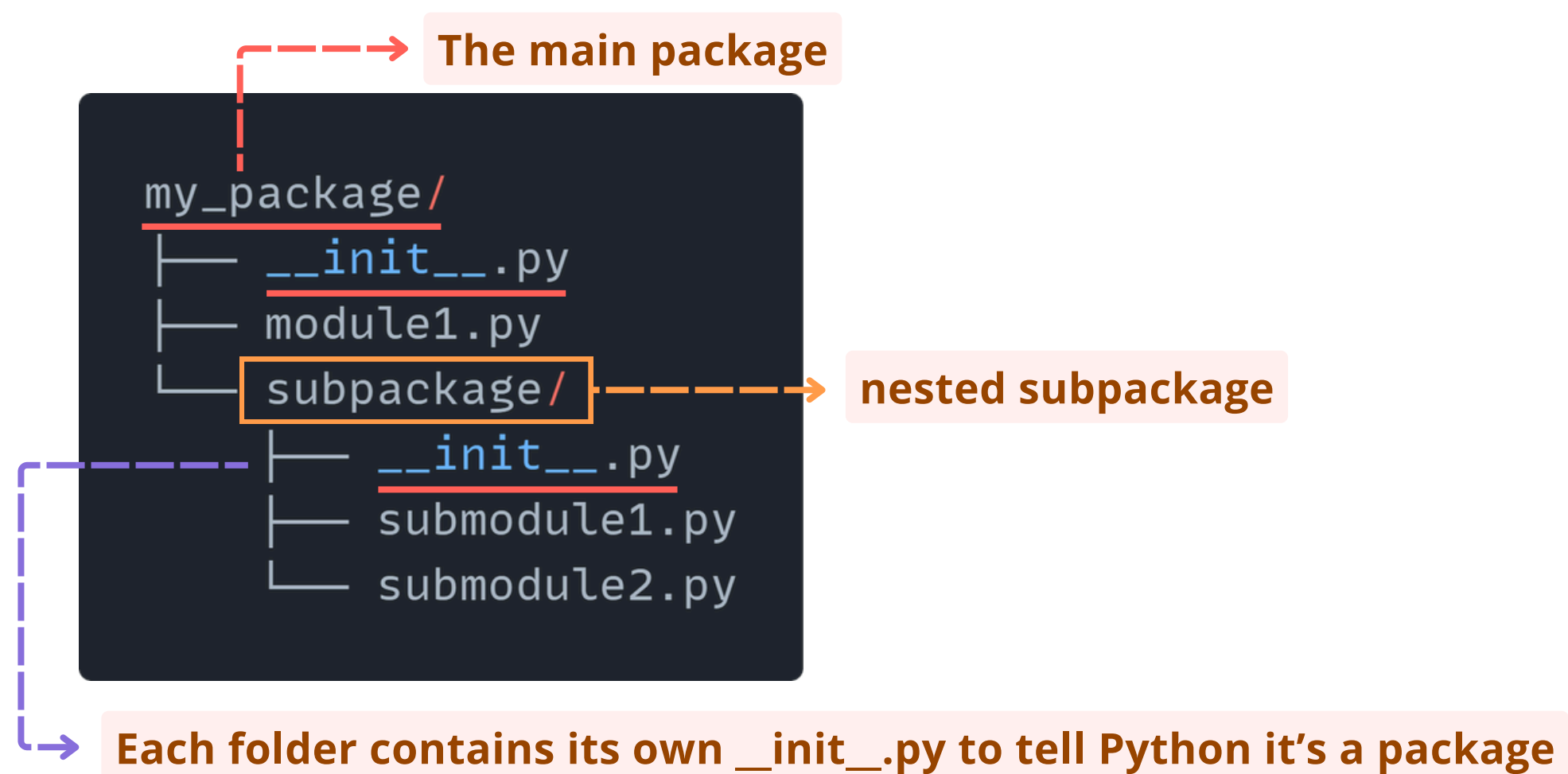
→ Shows only modules

→ Shows functions and variables

Multi-Level Packages

- A **multi-level package** (also called a **nested package**) is a package that contains **one or more subpackages** inside it.
- This structure **helps** you **organize large projects** especially when your codebase grows beyond a few modules.

Structure of a Multi-Level Package



subpackage/submodule2.py

```
def sub_farewell():
    return "Goodbye from Submodule 2!"
```

subpackage/init.py

```
print("Initializing subpackage")
```

subpackage/submodule1.py

```
def sub_greet():
    return "Hello from Submodule 1!"
```

Folder Structure

```
my_package/  
├── __init__.py  
├── module1.py  
└── subpackage/  
    ├── __init__.py  
    ├── submodule1.py  
    └── submodule2.py
```

main.py

```
>  main.py  
import sys  
  
sys.path.append(r"C:\Users\pralhad\Desktop\dev")  
  
from my_package.subpackage import submodule1, submodule2  
  
print(submodule1.sub_greet())  
print(submodule2.sub_farewell())  
  
# Output:  
# Initializing my_package  
# Initializing subpackage  
# Hello from Submodule 1!  
# Goodbye from Submodule 2!
```

use dot notation to reference nested modules.

print from main package level init file

print from subpackage level init file

Key Points

- Each **subpackage** must have its own **`__init__.py`** file to be recognized as a Python package.
- Use **dot notation** (**`subpackage.submodule`**) to reference nested modules.
- **Multi-level** packages allow **better code** organization for complex projects.

Flat vs Multi-Level Packages

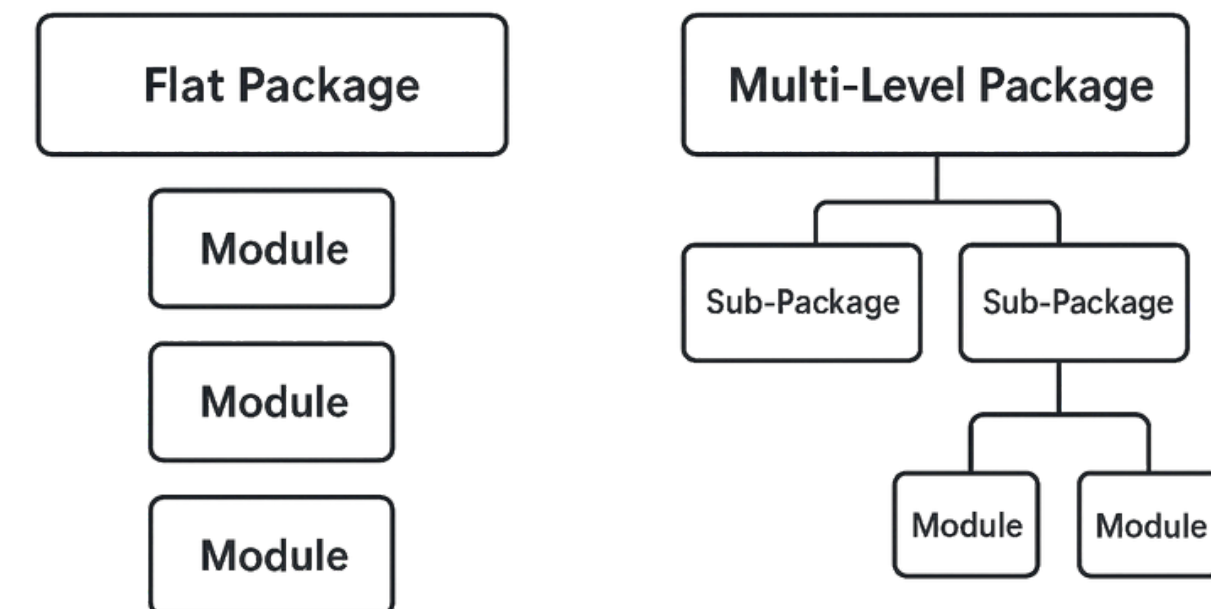
When structuring a Python project, we typically choose between:

- **A Flat Package** — where all **modules live** at the **same level**
- **A Multi-Level Package** — where related **modules are grouped** into **subpackages**

Each structure works well in different situations. Our choice depends on factors like:

- Project size
- Code complexity
- Whether the codebase needs to scale over time

Flat vs Multi-Level Packages



Flat Package

- A **Flat Package** has all its modules at the **same level** in a **single directory**.
- It's ideal for **small to moderately-sized projects** with **limited functionality**.

Structure of a Flat Package

```
flat_package/  
├── __init__.py  
├── module1.py  
├── module2.py  
└── main.py
```

Key Observations

- **Simple structure** — all modules are at the same level
- Easy to understand and **maintain** for **small projects**
- Imports are **straightforward**: `from flat_package import module1`
- Can become difficult to manage as functionality grows

| Ideal for tools or scripts with **limited scope** and **few modules**

Multi-Level Package

- A **Multi-Level Package** organizes modules into **subpackages**, grouping related functionalities together.
- It's the preferred structure for **larger, scalable, or team-based projects**.

Structure of a Multi-Level Package

```
multi_level_package/  
├── __init__.py  
├── module1.py  
└── subpackage/  
    ├── __init__.py  
    ├── submodule1.py  
    └── submodule2.py  
└── main.py
```

Key Observations

- Modules are grouped by functionality inside subpackages
- Imports use dot notation:
 - `from multi_level_package.subpackage import submodule1`
- **Easier to navigate** and **scale in larger projects**
- Slightly complex import paths, but **much more maintainable** in the long term

Ideal for tools with **many modules**, **team-shared libraries**, or **production-scale pipelines**

When to Use Each

Package Type	Best For	Pros	Cons
• Flat Package	→ Small utilities, single-purpose tools	→ Simple, direct imports	→ Becomes cluttered over time
• Multi-Level Package	→ Larger projects, shared libraries	→ Scalable, organized, modular	→ Requires more structure/setup

Comparison Table

Aspect	Flat Package	Multi-Level Package
• Structure	→ Modules at the same level	→ Organized into subpackages
• Scalability	→ Limited for large projects	→ Suitable for large projects
• Imports	→ Direct imports	→ Dot-notation imports
• Code Readability	→ Simple but cluttered with growth	→ Clear and modular
• Maintenance	→ Becomes harder over time	→ Easier to maintain
• Best For	→ Small projects	→ Large, complex projects

Avoid Hardcoding Paths when Importing

Using code like:

```
import sys
sys.path.append(r"C:\User\cus\lib")
```

Hardcoded Path

might work for **quick tests**, but it's **not recommended** for real projects.

Why Avoid It?

- **Not portable** → works only on **your machine**
- **Breaks easily** → fails on other systems or after folder changes
- **Hard to maintain** → paths need manual updates

What to Do Instead

1. Use Relative Paths (Dynamic)

- Use the **os module** to build paths based on your **script's location**:

```
import os, sys
base_path = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "lib"))

if base_path not in sys.path:
    sys.path.append(base_path)
```

WE SHALL LEARN THIS IN FUTURE

2. Package Your Code

Turn your **custom code** into a **package** and **install** it:

```
pip install -e .
```

This allows **clean imports**:

```
from my_package import module1
```

3. PYTHONPATH

- Use a **virtual environment**
- Or set a **PYTHONPATH** environment variable

These approaches keep scripts clean and portable.

sys.path.append() and sys.path.insert()

- `sys.path.append()`
- `sys.path.insert()`
- `sys.path.remove()`
- When to Use `sys.path.insert()` vs `sys.path.append()`

sys.path.append()

How It Works

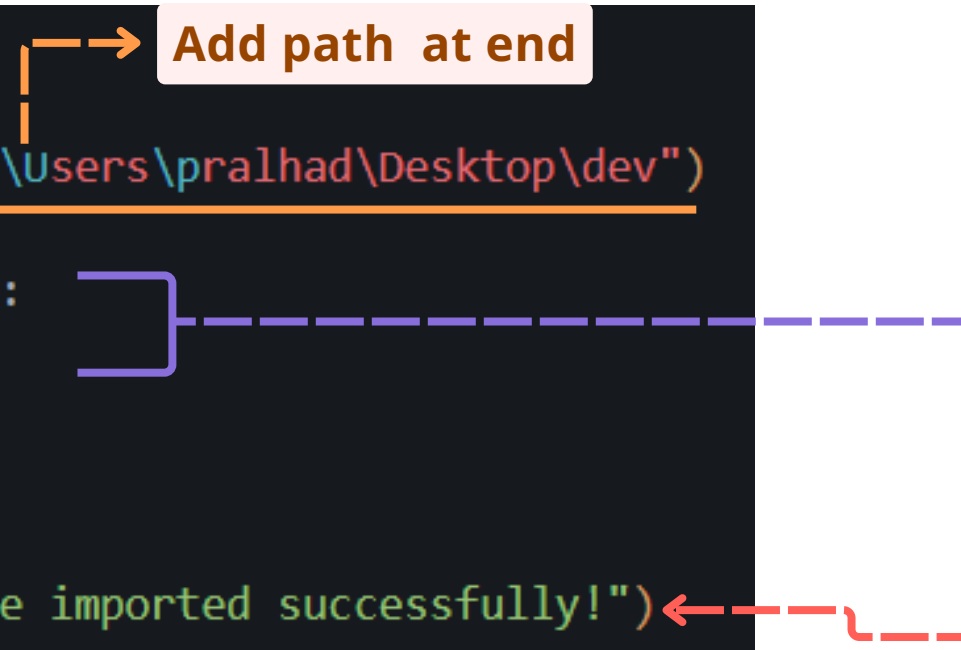
- Adds the specified path to the **end** of the sys.path list.
- Python will **only search this path after checking** all existing default locations.

```
import sys
sys.path.append(r"C:\Users\pralhad\Desktop\dev")

for path in sys.path:
    print(path)

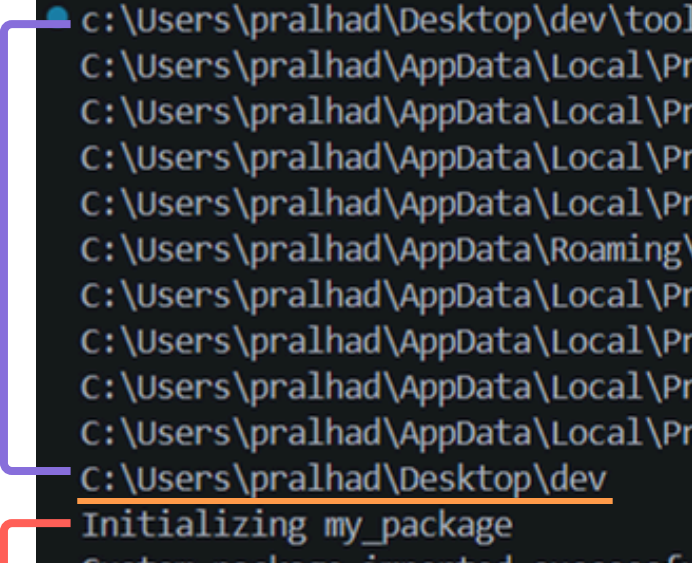
import my_package

print("Custom package imported successfully!")
```



Add path at end

```
PS C:\Users\pralhad\Desktop\dev> & C:/Users/pralhad/AppData/Local/Programs/Python/Python39/python.exe C:/Users/pralhad/Desktop/dev/tools/my_package.py
c:\Users\pralhad\Desktop\dev\tools
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\python39.zip
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\DLLs
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib
C:\Users\pralhad\AppData\Local\Programs\Python\Python39
C:\Users\pralhad\AppData\Roaming\Python\Python39\site-packages
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib\site-packages
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib\site-packages\win32
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib\site-packages\win32\lib
C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib\site-packages\Pythonwin
C:\Users\pralhad\Desktop\dev
Initializing my_package
Custom package imported successfully!
```



Order of Search:

0. Standard library,
1. site-packages,
2. other entries...,
3. '/path/to/my_custom_package'

Python searches in these order



sys.path.insert()

How It Works

- Inserts the **specified path** at a **given index** in **sys.path**.
- When inserted at **index 0**, the path gets **highest priority** — it's searched before all others.

Inserts custom path at the beginning of sys.path

```
> main.py
import sys

sys.path.insert(0, "C:/Users/pralhad/Desktop/dev")

import my_package

print("Custom package imported with priority!")
```

"Gives Dev folder highest priority during imports"

Order of Search:

0. '/path/to/my_custom_package',

1. Standard library,

2. site-packages, ...]

→ Highest Priority

When to Use It ?

- When you want to **override an existing module** or package of the same name.
- When your custom package must be found **first**.
- Helpful in development environments or when mocking dependencies.



what if we want to remove a sys.path ?

sys.path.remove()

Once a **custom path** is added, you might want to remove it after use — especially in temporary scripts.

```
import sys
import os

custom_path = r"C:\Users\pralhad\Desktop\dev"

if custom_path not in sys.path:
    sys.path.insert(0, custom_path)

print(custom_path in sys.path) # True

import my_package

print("Custom package imported")

sys.path.remove(custom_path)

print(custom_path in sys.path) # Output: False
```

Adds the path

Imports the package from that path

removes the specified path

Confirms it's removed

Common reasons include:

- Preventing name collisions
- Switching between package versions
- Keeping the environment clean after a one-time import

→ If the path is not found, it raises a **ValueError**.

Verifying Module Import Location

Let's say you have:

- One version of `cus_mod.py` inside: `C:\Users\pralhad\Desktop\dev`
- Another version installed globally in **site-packages**

If you do:

```
import sys
sys.path.insert(0, r"C:\Users\pralhad\Desktop\dev")
import cus_mod
print(cus_mod.__file__)
```

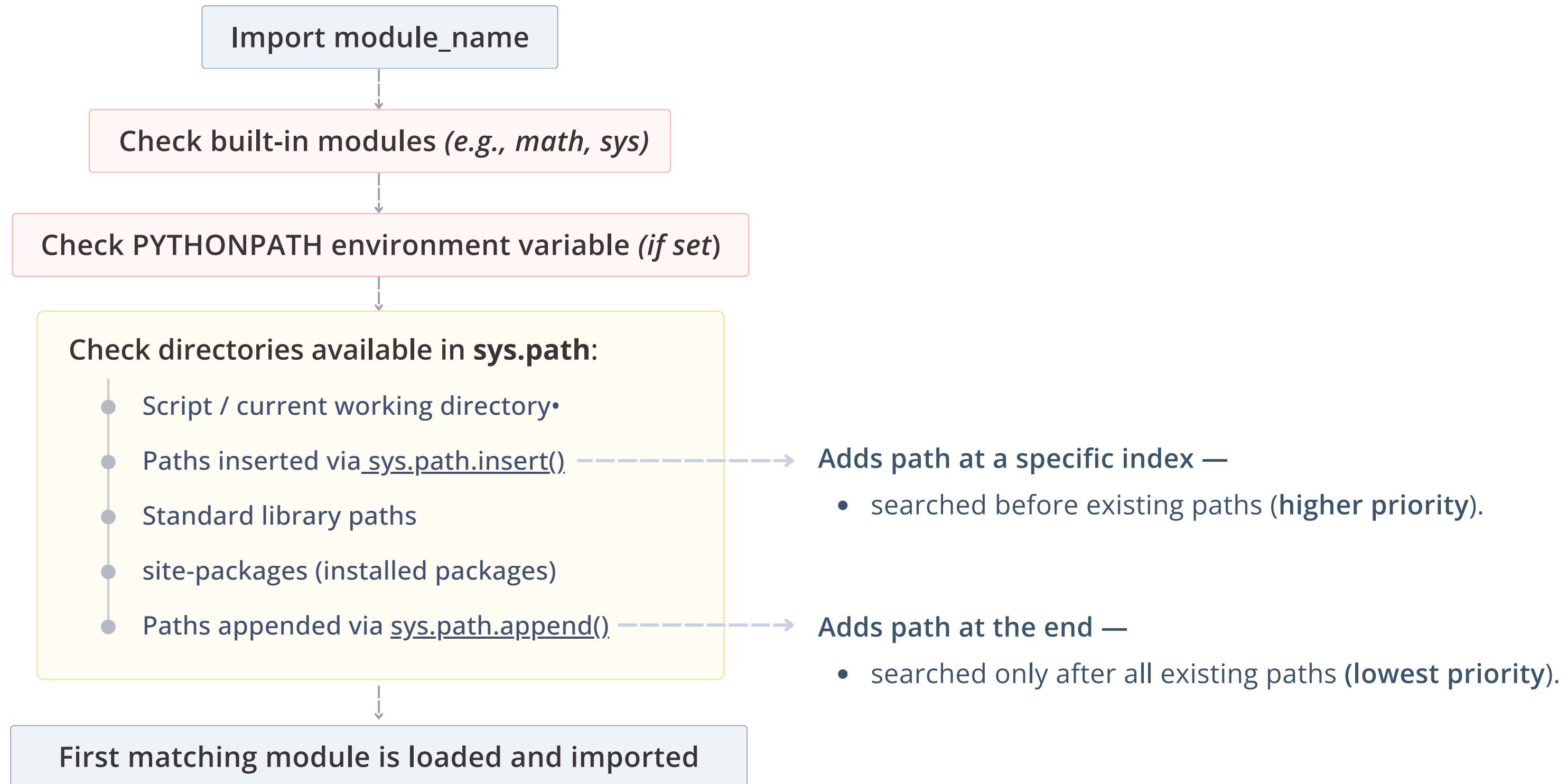
Diagram annotations:

- Insert Path at 0th Poston** (Note: misspelled as "Poston" in image) points to the `0` in `sys.path.insert(0, ...)`.
- File Path** points to the string `r"C:\Users\pralhad\Desktop\dev"`.
- Check path** points to the `__file__` attribute in `print(cus_mod.__file__)`.

Python will show: `C:\Users\pralhad\Desktop\dev\cus_mod.py`

This confirms it's importing the module from the **custom development path**, not the global one.

Python Module Import Search Flow



When to Use `sys.path.insert()` vs `sys.path.append()`

- **Use `sys.path.insert(0, ...)` when:**

- You want your custom package to take precedence over default or installed packages.
- Avoiding module naming conflicts.

- **Use `sys.path.append()` when:**

- The custom path is not critical and can wait until default paths are searched.
- You want to minimize the risk of overriding standard library modules.

Comparison Table

Feature	<code>sys.path.append()</code>	<code>sys.path.insert(0, ...)</code>
• Position	→ Adds path at the end	→ Adds path at the beginning
• Search Priority	→ Low Priority	→ High Priority
• Use Case	→ Non-critical modules	→ Critical modules or to avoid name conflicts
• Risk of Overriding	→ Unlikely	→ Possible (if names clash with built-in modules)
• Example Order	→ [..., '/custom_path']	→ ['/custom_path', ...]

Why Custom Python Packages in VFX?

- Modularity and Reusability
- Automation of Repetitive Tasks
- Scalability and Maintainability
- Pipeline Consistency
- Benefits of Using Custom Packages in a VFX Pipeline

Modularity and Reusability

Problem:

Artists and TDs frequently need to perform repetitive tasks like

- file management, asset importing/exporting, or render configuration.

Solution: Custom Python packages allow these tasks to be **modularized into reusable tools**.

Example: Asset Publishing Tool

```
# asset_manager.py  
  
def publish_asset(asset_name, version):  
    """Publish an asset with versioning."""  
    print(f"Asset '{asset_name}' published as version {version}")
```

Defines reusable function

Usage:

```
# main.py  
  
from my_pipeline.asset_manager import publish_asset  
  
publish_asset("dragon_model", "v1.0")
```

Import function from package

Benefit: The same function can be reused across multiple projects and teams without duplication.

Automation of Repetitive Tasks

Problem:

Repetitive tasks like **file renaming**, **folder structuring**, or **render job** submissions are time-consuming and error-prone.

Solution:

Automate these tasks with custom scripts.

Example: Folder Structure Automation

```
import os                                     # folder_setup.py

def create_project_structure(project_name):
    os.makedirs(f"{project_name}/assets", exist_ok=True)
    os.makedirs(f"{project_name}/renders", exist_ok=True)
    print(f"Project structure created for '{project_name}'")
```

Usage:

```
from my_pipeline.folder_setup import create_project_structure
create_project_structure("epic_dragon_scene")
```

Import reusable function

Benefit: Eliminates manual errors and saves valuable time for creative tasks.

Integration with VFX Software

Problem: Different VFX tools (Maya, Houdini, Nuke) need to communicate seamlessly.

Solution: Python acts as the glue language, and custom packages facilitate smooth communication between these tools.

Example: Automating Maya Scene Import

```
import maya.cmds as cmds          # maya_tools.py

def import_asset(asset_path):
    """Import an asset into Maya."""
    cmds.file(asset_path, i=True)
    print(f"Asset imported from '{asset_path}'")
```

Usage:

```
from my_pipeline.maya_tools import import_asset
import_asset("/projects/dragon_model.mb")
```

Benefit: Direct integration with industry-standard software streamlines workflows.

Scalability & Maintainability

Problem: As VFX projects grow, the pipeline must handle more assets, shots, and teams without breaking down.

Solution: Custom packages provide scalable tools and centralize functionalities, making them easy to maintain.

Example: Global Render Settings

```
# render_settings.py  
def set_global_render_settings(resolution="1920x1080", samples=64):  
    print(f"Render resolution set to {resolution} with {samples} samples")
```

Usage:

```
from my_pipeline.render_settings import set_global_render_settings  
  
set_global_render_settings(samples=128)
```

Benefit: Changes to global render settings only need to be made in one place and propagate across the entire pipeline.

Pipeline Consistency

Problem: Each studio has its **unique workflows**. Without standardization, projects become chaotic.

Solution:

Custom packages enforce naming conventions, directory structures, and render parameters consistently across the pipeline.

Example: Naming Convention Validator

```
# validator.py
def validate_asset_name(name):
    if "_" not in name:
        raise ValueError("Invalid asset name. Use underscores (e.g., dragon_model_v1).")
    print(f"Asset name '{name}' is valid.")
```

Usage:

```
from my_pipeline.validator import validate_asset_name

validate_asset_name("dragon_model_v1")
validate_asset_name("dragonModel")
```

Benefit: Enforces studio standards and prevents downstream errors due to inconsistent naming.

Benefits of Using Custom Packages in a VFX Pipeline

Benefit	Description
• Time Savings	→ Automates repetitive tasks, reducing manual effort.
• Reduced Errors	→ Validation scripts ensure fewer human errors.
• Consistency	→ Standard workflows across all departments.
• Scalability	→ Can handle multiple shows and large datasets.
• Integration	→ Seamless integration with industry-standard tools.
• Flexibility	→ Packages can be updated independently.

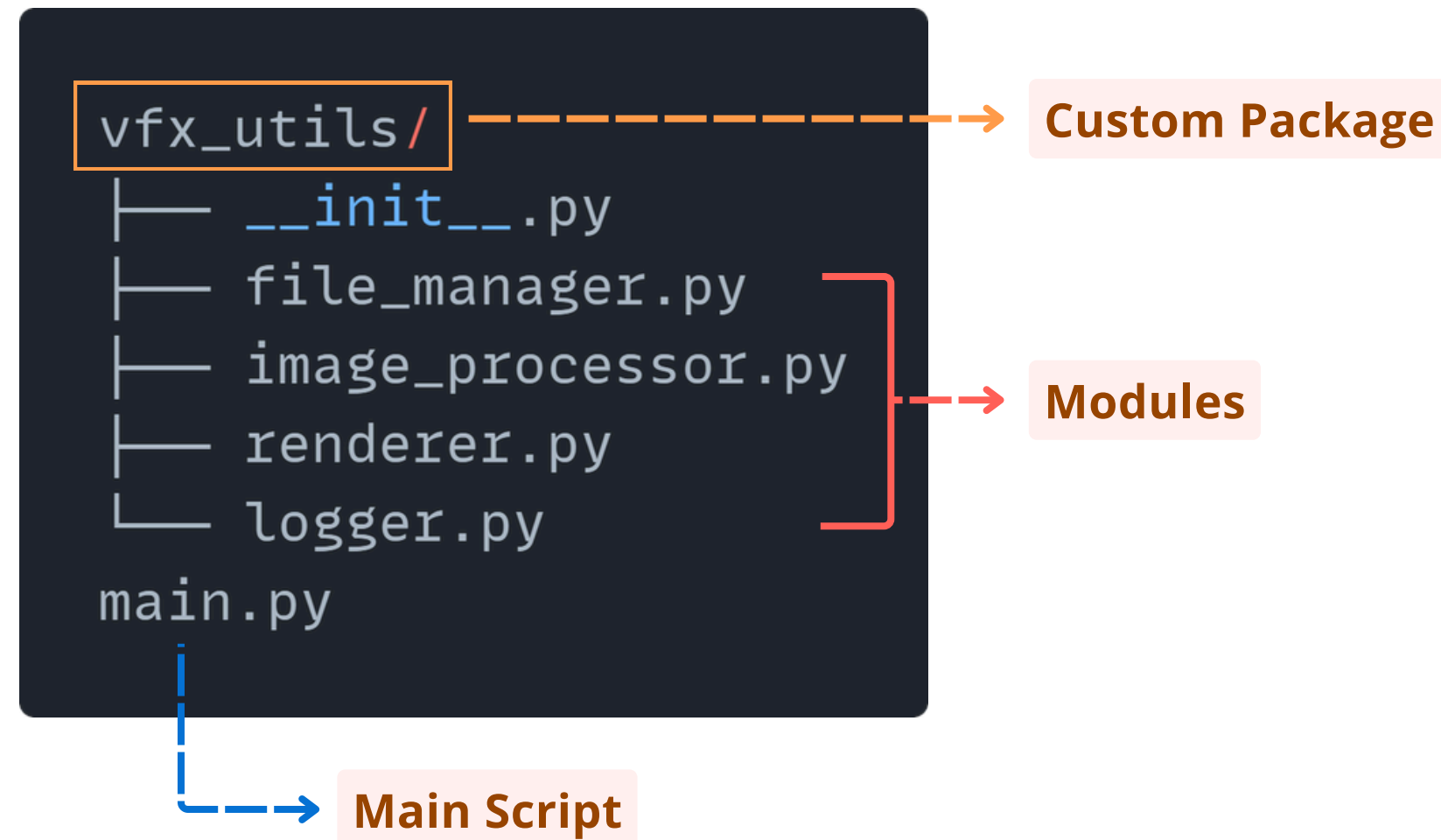
Real-World Examples

- A Custom Package for VFX Utilities
- Why Use Custom Packages?

A Custom Package for VFX Utilities

- Let's create a **custom** package for VFX utilities called **vfx_utils**.
- It will include modules that handle tasks like **file management, image processing, rendering, and logging**.

Folder Structure



file_manager.py

```
default_path = "/project/assets"

def list_files(path):
    return [f"{path}/file1.txt", f"{path}/file2.txt"]
def create_file(path, name):
    return f"Created file at {path}/{name}"
```

→ “Handles file listing and creation”

image_processor.py

```
supported_formats = ["jpg", "png", "exr"]

def resize_image(image_path, width, height):
    return f"Resized {image_path} to {width}x{height}"
def apply_filter(image_path, filter_name):
    return f"Applied {filter_name} filter to {image_path}"
```

→ “Processes and filters images”

renderer.py

```
default_render_engine = "Cycles"

def start_render(scene):
    return f"Rendering scene: {scene} with {default_render_engine}"
def stop_render():
    return "Render stopped"
```

→ “Controls scene rendering”

logger.py

```
log_file = "vfx_log.txt"

def log_info(message):
    return f"INFO: {message}"
def log_error(message):
    return f"ERROR: {message}"
```

→ “Logs info and error messages”

__init__.py

```
print("Initializing vfx_utils package")

from .file_manager import list_files, create_file
from .image_processor import resize_image, apply_filter
from .renderer import start_render, stop_render
from .logger import log_info, log_error
```

→ “Initializes the vfx_utils package and links all modules”

main.py

```
import sys
import os

package_path = os.path.abspath("path/to/vfx_utils")
if package_path not in sys.path:
    sys.path.append(package_path)

from vfx_utils import (
    list_files, create_file,
    resize_image, apply_filter,
    start_render, stop_render,
    log_info, log_error
)

print(list_files("/project/assets"))
print(create_file("/project/assets", "new_asset.txt"))

print(resize_image("image.jpg", 1920, 1080))
print(apply_filter("image.jpg", "Sepia"))

print(start_render("Scene_01"))
print(stop_render())

print(log_info("All systems operational"))
print(log_error("Render engine crashed"))
```

Import VFX utility functions

"List project files"

Resize image to 1080p

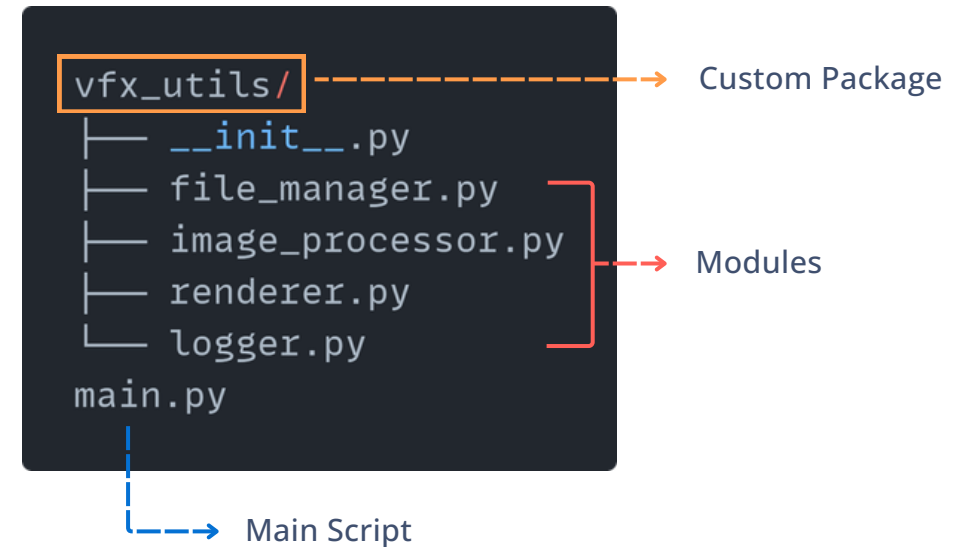
Apply Sepia filter

Stop rendering

Log info message

Log error message

Folder Structure



Expected Output:

```
Initializing vfx_utils package
['/project/assets/file1.txt', '/project/assets/file2.txt']
Created file at /project/assets/new_asset.txt
Resized image.jpg to 1920x1080
Applied Sepia filter to image.jpg
Rendering scene: Scene_01 with Cycles
Render stopped
INFO: All systems operational
ERROR: Render engine crashed
```

First line comes from the `__init__.py` file.

Key Takeaways

- Adding imports to `__init__.py` enables cleaner and more intuitive imports at the **package level**.
- Structuring your code into logical modules makes large codebases more **maintainable**.
- `sys.path` ensures Python knows **where to look** for your custom packages.

Why Use Custom Packages?

Organize large codebases:

- When your **project grows**, it's much easier to manage if you **split** related code into **separate modules** and **group** them **inside packages**.

Reuse modules across multiple projects:

- Once you've written a good set of **utility functions**, you don't want to keep **rewriting** them. Packages let you **import** and **reuse** your **own work**.

Improve readability and maintainability:

- When your code is **structured cleanly**, it's **easier** for you (or your **teammates**) to **read, understand**, and **update** it later.

Basic Arithmetic Module

Folder Structure:

```
arithmetic/  
├── add.py  
└── subtract.py  
main.py
```

File Content:

- arithmetic/add.py

```
def add_numbers(a, b):  
    return a + b
```

- arithmetic/subtract.py

```
def subtract_numbers(a, b):  
    return a - b
```

- main.py

```
from arithmetic import add, subtract  
  
print("Addition:", add.add_numbers(10, 5))  
# Output: 15  
  
print("Subtraction:", subtract.subtract_numbers(10, 5))  
# Output: 5
```


Create Custom Greeting Package

Folder Structure:

```
greetings/  
├── __init__.py  
├── english.py  
└── spanish.py  
main.py
```

File Content:

- greetings/__init__.py

```
# Empty file to mark it as a package
```

- main.py

```
from greetings import english, spanish  
  
print(english.greet_in_english("John"))  
# Output: Hello, John!  
  
print(spanish.greet_in_spanish("Maria"))  
# Output: Hola, Maria!
```

- greetings/english.py

```
def greet_in_english(name):  
    return f"Hello, {name}!"
```

- greetings/spanish.py

```
def greet_in_spanish(name):  
    return f"Hola, {name}!"
```

Temperature Conversion Library

Folder Structure:

```
temperature_conversion/  
├── __init__.py  
├── celsius.py  
└── fahrenheit.py  
main.py
```

File Content:

- temperature_conversion/__init__.py

```
# Package initialization  
print("Temperature conversion library initialized")
```

- temperature_conversion/fahrenheit.py

```
def to_celsius(f):  
    return (f - 32) * 5/9
```

- temperature_conversion/celsius.py

```
def to_fahrenheit(c):  
    return (c * 9/5) + 32
```

- main.py

```
from temperature_conversion import celsius, fahrenheit  
  
print(celsius.to_fahrenheit(25)) # Output: 77.0  
print(fahrenheit.to_celsius(77)) # Output: 25.0
```

Multi-Level Package for Shape Calculations

Folder Structure:

```
geometry/  
├── __init__.py  
├── circle.py  
└── shapes/  
    ├── __init__.py  
    ├── rectangle.py  
    └── triangle.py  
main.py
```

File Content:

- geometry/__init__.py

```
# Initialization for geometry package
```

- geometry/shapes/__init__.py

```
# Initialization for shapes subpackage
```

- geometry/shapes/triangle.py

```
def area(base, height):  
    return 0.5 * base * height
```

- geometry/circle.py

```
import math  
def area(radius):  
    return math.pi * radius ** 2
```

- geometry/shapes/rectangle.py

```
def area(length, breadth):  
    return length * breadth
```

- main.py

```
from geometry import circle  
from geometry.shapes import rectangle, triangle  
  
print("Circle Area:", circle.area(5))           # Output: 78.54  
print("Rectangle Area:", rectangle.area(4, 5))  # Output: 20  
print("Triangle Area:", triangle.area(3, 6))    # Output: 9.0
```

Simple File Organizer Module

A module to list and organize files in a directory.

Folder Structure:

```
file_organizer/  
├── __init__.py  
├── list_files.py  
└── create_folders.py  
main.py
```

File Content:

- file_organizer/__init__.py

```
# File Organizer Package Initialization
```

- file_organizer/create_folders.py

```
import os  
  
def create_folder(directory, folder_name):  
    os.makedirs(os.path.join(directory, folder_name), exist_ok=True)  
    return f"Folder '{folder_name}' created in '{directory}'"
```

- main.py

```
from file_organizer import list_files, create_folders  
  
print("Files:", list_files.list_files_in_dir("."))  
print(create_folders.create_folder(".", "new_folder"))
```

- file_organizer/list_files.py

```
import os  
  
def list_files_in_dir(directory):  
    return os.listdir(directory)
```

Task Manager Package

Folder Structure:

```
task_manager/  
├── __init__.py  
├── add_task.py  
└── list_tasks.py  
main.py
```

File Content:

- task_manager/__init__.py

```
tasks = []
```

- task_manager/list_tasks.py

```
from task_manager import tasks  
  
def get_tasks():  
    return tasks
```

- task_manager/add_task.py

```
from task_manager import tasks  
  
def add_task(task):  
    tasks.append(task)  
    return f"Task added: {task}"
```

- main.py

```
from task_manager import add_task, list_tasks  
  
add_task.add_task("Complete Python homework")  
add_task.add_task("Review notes")  
  
print("To-Do List:")  
print(list_tasks.get_tasks())
```


Package for String Manipulations

Folder Structure:

```
string_utils/  
├── __init__.py  
├── reverse.py  
└── capitalize.py  
main.py
```

File Content:

- string_utils/__init__.py

```
print("String Utilities Package Initialized")
```

- string_utils/reverse.py

```
def reverse_string(s):  
    return s[::-1]
```

- string_utils/capitalize.py

```
def capitalize_words(s):  
    return s.title()
```

- main.py

```
from string_utils import reverse, capitalize  
  
print(reverse.reverse_string("hello"))  
# Output: olleh  
  
print(capitalize.capitalize_words("hello world"))  
# Output: Hello World
```


package to handle file reading & writing

Folder Structure:

```
file_ops/  
├── __init__.py  
├── read.py  
└── write.py  
main.py
```

File Content:

- file_ops/__init__.py

```
print("File Operations Package Initialized")
```

- file_ops/read.py

```
def read_file(filepath):  
    with open(filepath, 'r') as file:  
        return file.read()
```

- file_ops/write.py

```
def write_file(filepath, content):  
    with open(filepath, 'w') as file:  
        file.write(content)  
    return "File written successfully"
```

- main.py

```
from file_ops import read, write  
  
write.write_file("example.txt", "This is a test.")  
  
print(read.read_file("example.txt"))  
# Output: This is a test.
```

Package for Unit Conversion

Folder Structure:

```
unit_conversion/  
├── __init__.py  
├── length.py  
└── weight.py  
main.py
```

File Content:

- unit_conversion/__init__.py

```
print("Unit Conversion Package Initialized")
```

- unit_conversion/length.py

```
def meters_to_kilometers(m):  
    return m / 1000
```

- unit_conversion/weight.py

```
def kilograms_to_grams(kg):  
    return kg * 1000
```

- main.py

```
from unit_conversion import length, weight  
  
print(length.meters_to_kilometers(1500))  
# Output: 1.5  
  
print(weight.kilograms_to_grams(2))  
# Output: 2000
```


Multi-Level Package for Math Utilities

Folder Structure:

```
math_utils/  
├── __init__.py  
├── basic/  
│   ├── __init__.py  
│   ├── addition.py  
│   └── subtraction.py  
└── advanced/  
    ├── __init__.py  
    ├── factorial.py  
    └── power.py  
main.py
```

File Content:

- math_utils/__init__.py

```
print("Math Utilities Package Initialized")
```

- math_utils/basic/subtraction.py

```
def subtract(a, b):  
    return a - b
```

- math_utils/advanced/power.py

```
def power(base, exponent):  
    return base ** exponent
```

- math_utils/basic/addition.py

```
def add(a, b):  
    return a + b
```

- math_utils/advanced/factorial.py

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)
```

- main.py

```
from math_utils.basic import addition, subtraction  
from math_utils.advanced import factorial, power  
  
print(addition.add(10, 5)) # Output: 15  
print(subtraction.subtract(10, 5)) # Output: 5  
  
print(factorial.factorial(5)) # Output: 120  
print(power.power(2, 3)) # Output: 8
```

Thank You

